

What Does a Repair Statistic Measure? A Claim-Indexed Audit of Repository-Level LLM Repair

Jiangwei Xue
Independent Researcher
jw.xue@lucentia.company

Zhida Qin
Independent Researcher
shin_robotics@akane.waseda.jp

Yuda Bi*
TReNDS Center (GSU, GaTech, Emory)
yudabi95@gmail.com
*Corresponding author.

DOI: 10.5281/zenodo.22668842

Abstract

Repository-level repair systems emit many logged events before a verifier pass. A rate over one event is easy to misread when the report omits the event’s operational meaning, opportunity budget, support unit, or verifier boundary. We introduce a claim-indexed measurement model that binds each repair statistic to five coordinates: construct, opportunity design, unit of support, endpoint interpretation, and instrumentation provenance. We instantiate the model in a retrospective audit of 2,400 executions: two 30-task DeepSeek strata with disjoint task IDs and a 30-task GLM boundary stratum that reuses the clean task surface. A field-integrity audit identifies 12 raw GLM action positives that conflict with the patch-application evidence required by the governed-action construct. The harmonized table contains no verifier pass outside the governed-action state, while conditional endpoint yield is 14/199 (7.04%) and 14/204 (6.86%) in the two DeepSeek strata. Under eight retained opportunities per fixed cell, governed-action discovery reaches 40.83% and 46.67%; verifier-pass discovery reaches 4.17% and 1.67%. The 28 passing rows reduce to seven positive cells, three tasks, and two task families. Endpoint controls produce the expected result for all six gold runs, six no-op runs, and nine reference-derived mutants. These findings show that repair evidence cannot be reduced to one success scalar. Field semantics, opportunity, support, verifier scope, and instrumentation determine what a reported number measures and which comparisons it can sustain.

Keywords: repository-level software repair; coding agents; construct validity; evaluation methodology; repeated sampling; support concentration; test oracle; reproducibility

1 Introduction

One retained repair stratum in this study contains 204 governed actions among 960 executions. At an eight-opportunity budget, 56 of 120 fixed cells expose at least one governed action. Across the full retained stratum, 26 of 30 tasks support at least one governed action. The same stratum contains 14 verifier-passing executions. Those passes come from two cells of one task and one task family. Each statement is accurate. They describe different constructs at different aggregation levels.

Repository-level repair produces a long path from model output to an accepted patch. A response must satisfy an interface, specify an edit, apply to a particular revision, remain within the allowed action surface, enter an execution environment, and satisfy a task-specific verifier. Modern coding-agent benchmarks retain many of these events because they help explain failures and guide system development [12, 39, 43, 44]. The final report often compresses them into a small set of rates. Compression becomes a measurement problem when the name of the rate leaves its construct, budget, or support implicit.

The problem begins before aggregation. A stored field is an output of a runner and scorer, not a self-validating fact. In the supplementary GLM records studied here, 12 rows carry a positive historical action flag while the retained patch-application check records failure. The paper’s governed-action definition requires that check to succeed. Counting the raw flag would therefore include rows that do not meet the stated construct. The discrepancy changes support at several levels: positive rows fall from 98 to 86, positive cells from 29 to 27, and positive tasks from 20 to 19.

A valid upstream construct still leaves a criterion question. A governed action establishes that a non-empty, allowed source change reached the strict application surface. It does not establish that the verifier passed. In the two DeepSeek strata, only 14 of 199 and 14 of 204 governed actions reach the endpoint. Repetition then changes what is likely to be observed within a fixed cell: action discovery rises substantially across one to eight retained opportunities, while pass discovery remains sparse. Moving from execution rows to cells, tasks, and task families reveals a much narrower support base.

The endpoint has its own interpretation boundary. A verifier pass records satisfaction of a frozen executable criterion. Software testing and automated program repair have long distinguished test plausibility from complete correctness [1, 16, 22, 31]. This study checks the local verifier with reference, no-op, and near-reference negative controls. The controls establish discrimination on the tested cases; semantic adequacy remains a broader construct.

We organize these issues with a *claim index*. For a reported estimate $\hat{\theta}$, the index records

$$\mathcal{I}(\hat{\theta}) = \langle C, K, U, V, \Pi \rangle.$$

Here C is the operational construct, K is the opportunity and selection design, and U is the support unit. V states the strongest interpretation licensed by the observed criterion. Π identifies the task, runner, scorer, model/provider, and artifact stratum that produced the measurement. The index exposes which measurement conditions agree and which differ. A contrast that attributes a difference to one coordinate must match or model the remaining coordinates. This formulation gives a concrete rule for defining, auditing, and comparing repair results.

We evaluate the model on 2,400 retained executions arranged as 300 fixed cells with eight repetitions each. The two DeepSeek task pools contain 30 task instances apiece and share no exact task IDs. The GLM boundary stratum reuses the 30 clean-pool task IDs under a separate hosted-model context. The design supports two forms of pattern triangulation: a task-surface shift within the DeepSeek harness context and a hosted-model boundary on a matched task surface. Each stratum retains its own denominator.

The paper makes three contributions.

1. We define a claim-indexed model of repair evidence with three measurement boundaries—field to construct, construct to verifier, and verifier to semantic adequacy—plus two orthogonal aggregation dimensions: opportunity and support.
2. We apply the model to a 2,400-execution audit and quantify four linked effects: field-level construct conflicts, low conditional endpoint yield, asymmetric accumulation under repeated opportunities, and concentration of endpoint evidence across cells, tasks, and task families.
3. We provide a reproducible audit protocol that preserves raw fields, derives versioned harmonized constructs, checks the endpoint with positive and negative controls, and links claim-bearing tables to row- and provider-level provenance.

The resulting account treats intermediate signals as measurements with defined roles. It preserves their diagnostic value while keeping the endpoint, budget, and breadth of evidence visible.

2 Related Work

2.1 From automated repair to repository-level agents

Automated program repair (APR) established test-guided patch generation as a research area well before repository-level language-model agents [7, 21]. Defects4J supplied reproducible real faults for controlled

Java studies [13]; subsequent cross-benchmark experiments showed that apparent repairability can vary substantially with the benchmark and execution framework [6, 46]. This history matters here because a repair statistic is conditional on both the task surface and the machinery that produces and validates candidates.

SWE-bench made executable issue resolution in real repositories a standard target for language-model evaluation [12]. SWE-agent, Agentless, AutoCodeRover, and OpenHands illustrate how interfaces, localization procedures, sandboxes, and orchestration shape the path from issue text to candidate patch [35, 39, 44, 50]. Later benchmarks broadened the surface to live issues, multiple languages, long-horizon upgrades, and paid freelance tasks [20, 25, 32, 43, 49]. SWE-smith further makes task construction and executable-environment generation explicit parts of the software-agent data pipeline [45]. Together, these systems expose many observable events before the final test result and make instrumentation part of the evaluation condition.

Recent work uses trajectories to recover process information omitted by resolve rate. AgentLens classifies passing trajectories by process quality and identifies “lucky pass” patterns [26]. TraceProbe normalizes tool trajectories and detects structural patterns such as search loops and skipped verification [29]. Cross-framework analysis shows that the association between a behavioral signal and resolution can change direction across agent configurations [19]. Process evaluation also depends on the level of analysis: action prediction, task uncertainty, and step-level causal contribution answer different questions [9].

These studies analyze what agents do over time. Our audit starts one level lower, at the measurement fields produced by the execution pipeline. It asks whether a logged field satisfies its stated invariant, then follows that construct to the verifier and through the available aggregation units.

2.2 From weak test oracles to endpoint adequacy

Test-based repair has long separated a patch that passes the available suite from a patch that satisfies the intended behavior. Generate-and-validate search spaces can contain orders of magnitude more test-passing incorrect patches than correct ones [18]. Empirical studies likewise document test-suite overfitting and show that repair reliability depends on the strength and composition of the test suite [22, 31, 48]. These results establish the first link in the historical chain relevant to our endpoint: a finite test oracle licenses a bounded operational claim.

Patch-correctness research then developed ways to probe that boundary. DiffTGen generates tests that distinguish candidate behavior from a reference implementation [40]. Behavior-similarity methods use executions to filter likely incorrect patches [41]. Comparative studies show that correctness assessors differ in precision, recall, and project sensitivity [16, 34]. Large-scale random testing against developer patches extends this assessment across repair tools while retaining an explicit reference-oracle assumption [47]. Metamorphic testing provides another route when useful input transformations and output relations can be specified [27]. In LLM code generation, EvalPlus shows that expanded tests can reject solutions accepted by the original benchmark and alter model rankings [17].

Repository-level work continues this endpoint audit. BSG-VA replays an agent’s validation command on buggy, candidate, and gold states [42]. Studies of agent-generated tests ask whether those tests support issue resolution or mainly provide feedback [5]. PatchDiff compares benchmark-passing patches with developer patches and reveals behavioral divergence after acceptance [36]. The broader verification literature describes executable rewards as proxies for user intent whose faithfulness and robustness can change with task and policy capability [33].

Our endpoint controls occupy a narrower role. They test whether the frozen verifier separates known-good, unrepaired, and selected near-reference candidates on the audited tasks. Reference-derived mutants are local probes rather than substitutes for the full distribution of real faults, a boundary established in mutation-testing research [14]. The controls therefore calibrate the endpoint used in this analysis without converting verifier pass into semantic adequacy.

2.3 Aggregation, protocol, measurement, and reproducibility

Pass@ k makes the number of sampled code candidates visible [4]. Recent work shows that this estimator can be misapplied when unit tests are treated as independent rollouts. Reliability@ k instead aggregates repeated complete attempts [11]. Our repeated-opportunity measure applies the same retained opportunities to governed action and verifier pass, allowing matched-budget comparison of their accumulation.

Evaluation validity also depends on task selection and protocol. LiveCodeBench uses temporal segmentation to reduce contamination and benchmark saturation [10]. HackDetect audits agent exploitation of exposed artifacts and invalid scoring paths [28]. An agent-safety benchmark audit shows that metric choice, target behavior, and model panel can change the meaning of a quoted score [37]. Broader analyses trace validity loss through task construction, simulation, and judgment [3].

The claim index is grounded in a longer software-measurement tradition. Property-based measurement work warns that different concepts can share one metric label unless the measured property is explicitly defined [2]. Empirical-software-engineering guidance likewise treats units, context, design, and validity as reportable parts of an empirical result [15, 23, 24, 30, 38]. Repository-mining research adds a provenance requirement: mutable sources, retrieval procedures, and archived derivatives determine whether a result can be reproduced [8]. Our five-coordinate index specializes these principles to repository-level repair by binding a statistic to its construct, opportunity design, support unit, endpoint interpretation, and instrumentation provenance.

Table 1: Adjacent evaluation boundaries. The present study connects field integrity, criterion relation, opportunity, support, and endpoint scope in one retained execution corpus.

Work line	Audited object	Primary unit	Central question
Trajectory diagnostics [26, 29]	Search, edit, tool, and validation sequence	Trajectory	What process structure accompanies a resolved or unresolved run?
Cross-framework behavior [19]	Behavioral feature and outcome association	Agent configuration	Does a signal retain its direction across frameworks?
Validation evidence [42]	Agent-executed validation command	Validation event	Does a positive check distinguish buggy, candidate, and gold states?
Post-pass correctness [36]	Benchmark-passing patch	Patch	Does a plausible patch match the behavior intended by the developer patch?
Repeated-rollout metrics [11]	Fully passing rollout under repeated attempts	Task-rollout set	Is repeated success aggregated with the intended sampling unit?
This study	Logged field, governed action, and verifier endpoint	Row, cell, task, family	Which claim is licensed after construct, budget, support, and endpoint are fixed?

Table 1 locates the present study among these lines. Its unit spans a nested evidence chain: raw field, harmonized action construct, verifier endpoint, repeated cell, task, and task family. The contribution lies in auditing those boundaries together and attaching each reported value to an explicit claim index.

3 A Claim-Indexed Model of Repair Evidence

A repository-repair evaluation observes a chain of proxies. The chain in this study is

$$\tilde{A}_i \xrightarrow{\mathcal{H}} A_i \longrightarrow P_i \dashrightarrow Y_i, \quad (1)$$

where $\tilde{A}_i$ is the historical action field, $\mathcal{H}$ is the documented harmonization rule, A_i is the governed-action construct, P_i is the frozen verifier-pass indicator, and Y_i denotes semantic adequacy. The solid arrows connect observed or derived variables. The dashed arrow marks an interpretive step that the study does not observe directly.

3.1 Three measurement boundaries

Field to construct. A field name such as `action_governance_success` expresses an intended meaning. Its scientific meaning comes from the rule that generated it and the invariants that support the rule. Runner changes, export mappings, missing artifacts, and event-order differences can break the link between a stored value and its intended construct. An encoding audit checks that link. The output is a versioned construct A_i that coexists with the immutable raw field $\tilde{A}_i$.

Construct to criterion. An upstream construct can be valid and still carry little information about a stronger endpoint. The criterion relation asks how often $A_i = 1$ coincides with $P_i = 1$, whether $P_i = 1$ appears outside the expected upstream state, and how the relation varies across the declared instrumentation strata. In this paper, governed action is a pipeline construct and verifier pass is the criterion against which its endpoint yield is measured.

Criterion to semantic adequacy. A verifier pass establishes that a finite executable criterion was met. Semantic adequacy includes the intended behavior, relevant untested inputs, and the absence of unacceptable regressions. Endpoint controls can test selected regions of this gap. They cannot observe the entire semantic construct. Keeping P_i and Y_i separate prevents a local test result from absorbing a broader meaning than the test surface provides.

3.2 The claim index

A scalar estimate suppresses the choices that produced it. Software-measurement theory cautions that the same label can denote different concepts unless the measured property and admissible interpretation are explicit [2]. We make those choices part of the reported object. For estimate $\hat{\theta}$,

$$\mathcal{I}(\hat{\theta}) = \langle C, K, U, V, \Pi \rangle. \quad (2)$$

Table 2 defines the coordinates. For two estimates, let

$$\mathcal{M}(\hat{\theta}_a, \hat{\theta}_b) = \{q \in \{C, K, U, V, \Pi\} : q(\hat{\theta}_a) \neq q(\hat{\theta}_b)\} \quad (3)$$

be their mismatch set. The set records which parts of the measurement question change. A direct contrast intended to isolate coordinate q^* requires $\mathcal{M}(\hat{\theta}_a, \hat{\theta}_b) \subseteq \{q^*\}$; a design may instead model additional mismatches explicitly. Descriptive multi-coordinate contrasts remain useful when their composite meaning is stated. For example, governed-action and verifier-pass discovery at $k = 8$ share the opportunity design, support unit, and instrumentation stratum; the construct and its interpretation boundary change together. Comparing action discovery at $k = 8$ with pass prevalence per execution also changes opportunity and unit, so the numerical gap combines additional questions.

Table 2: Coordinates of the claim index. A complete report fixes all five before interpreting an estimate.

Coordinate	Name	Question fixed by the coordinate	Example in this study
C	Construct	Which operational event counts as positive?	Harmonized governed action or verifier pass
K	Opportunity design	How many retained attempts are available, and how are they selected or aggregated?	One execution; or a size- k subset of eight cell repetitions
U	Support unit	Which units carry the positive evidence?	Execution row, fixed cell, task instance, task family
V	Interpretation boundary	What is the strongest interpretation licensed by the observed criterion?	Allowed applicable source activity; frozen-criterion satisfaction; or semantic adequacy
Π	Instrumentation stratum	Which task surface, runner, scorer, model/provider route, and artifact version produced the value?	Clean DeepSeek, extension DeepSeek, or matched-task GLM boundary

The index applies across metrics and supplies a compact specification for each one. A leaderboard score, a diagnostic action rate, and a task-level reliability measure can each be useful once their coordinates are stated.

3.3 Opportunity and support as orthogonal dimensions

Opportunity and support change an estimate in different ways. Increasing K gives a fixed cell more chances to expose a positive event. It answers a discovery question: what fraction of cells would contain at least one positive observation under a specified subset of the retained attempts? It does not add new task contexts.

Changing U asks how broadly the observed positives are distributed. Ten positive rows in one cell provide ten execution observations, one positive cell, one positive task, and one positive task family. These counts describe the same corpus at progressively broader units. We summarize them with a support profile

$$\Sigma_s(M) = \langle R_s(M), C_s(M), T_s(M), F_s(M) \rangle, \quad (4)$$

where M is a binary signal and the four entries count positive rows, cells, tasks, and task families in stratum s .

The two dimensions often move in opposite directions. More opportunities can make a positive event easier to observe within a cell. Broader support units can reveal that those positives occupy only a small part of the task surface. Reporting both prevents repeated observations from being mistaken for broad evidence.

3.4 Research questions

The framework yields four research questions:

RQ1: Field integrity. Does the retained raw action field satisfy the application invariants of the governed-action construct?

RQ2: Criterion relation. How does harmonized governed action relate to verifier pass at the execution level?

RQ3: Opportunity and support. How do governed action and verifier pass accumulate under one to eight retained opportunities, and how broadly are their positives supported across cells, tasks, and task families?

RQ4: Endpoint boundary. How does the frozen verifier behave on known-good, unrepaired, and near-reference negative controls?

4 Study Design and Measures

4.1 Evidence corpus

The corpus contains 2,400 historical executions grouped into three frozen strata (Table 3). The DeepSeek clean stratum combines 30 tasks, two hosted model routes, and two protocol arms. Each task–route–arm cell has eight repetitions, yielding $30 \times 2 \times 2 \times 8 = 960$ rows. The extension stratum has the same factorial structure over a second 30-task pool. Its task IDs are disjoint from the clean pool; 11 task-family identifiers occur in both pools.

The GLM boundary stratum reuses the 30 clean-pool task IDs with one hosted model route and the same two protocol arms, yielding $30 \times 1 \times 2 \times 8 = 480$ rows. The corpus therefore supplies a partial-overlap design. Clean-to-extension changes the task surface while retaining the general DeepSeek harness context. Clean-to-GLM holds the task IDs fixed and changes the hosted-model context. The strata support pattern triangulation; each keeps its own descriptive denominator.

The task pools were frozen, admission-filtered RoadmapBench-derived sets spanning five implementation languages. They are study surfaces rather than probability samples from all repository-repair tasks. Cross-benchmark APR results and temporally constructed code benchmarks both show why the sampling surface must remain part of the claim [6, 10]. The retained metadata includes task-family identifiers. A reliable repository identifier is absent, so support is reported through the family level.

Table 3: Study strata. Tasks/families reports the number of task instances and retained task-family identifiers. Each cell contains eight model executions.

Stratum	Task relation	Routes	Tasks/families	Cells $\times$ reps.	Rows
DeepSeek clean	Base task surface	2	30 / 14	120×8	960
DeepSeek extension	Task-ID-disjoint extension	2	30 / 13	120×8	960
GLM boundary	Same task IDs as clean	1	30 / 14	60×8	480

4.2 Repair protocol

The frozen repair interface asks the model for a structured state and explicit edit operations. Write operations modify or create source files; delete operations remove source files. The evaluator applies the operations sequentially to a working copy. Exact anchors must match. The application layer performs no fuzzy matching, implicit path repair, silent skip, or unreported whole-file replacement. A failed required operation ends application for that candidate.

Two historical prompt arms, `single_shot_v3` and `staged_v4`, appear in every stratum. The DeepSeek strata also contain two route labels, `deepseek-v4-flash` and `deepseek-v4-pro`. These labels define recorded measurement conditions. Route-by-arm summaries appear in Section C; the main analysis aggregates within each frozen stratum.

4.3 Raw and harmonized variables

Let $\tilde{A}_i$ indicate a positive value in the historical `action_governance_success` field. Let E_i indicate `deterministic_edit_apply_success=1`, and let G_i indicate `patch_apply_check_ok=1`. The harmonized governed-action construct is

$$A_i = \mathbf{1}(\tilde{A}_i = 1 \wedge E_i = 1 \wedge G_i = 1). \quad (5)$$

The raw action flag supplies the non-empty, allowed action decision; E_i and G_i enforce the retained application evidence. The retained derived field implements Equation (5). Raw values remain unchanged.

Verifier pass is

$$P_i = \mathbf{1}(\text{answer_success}_i = 1). \quad (6)$$

These indicators count observed positive events among all planned executions. A row without a positive record contributes no positive event; the indicator does not assign a semantic cause to that row. Earlier pipeline fields such as state validity and partial reward are retained where available, but their historical coverage is not consistent across all three strata. Cross-stratum estimands therefore use A_i and P_i .

4.4 Execution prevalence and conditional endpoint yield

For stratum s with N_s rows, execution prevalence is

$$\hat{p}_{M,s} = \frac{1}{N_s} \sum_{i \in s} M_i, \quad M \in \{A, P\}. \quad (7)$$

The relation from governed action to verifier pass is summarized by conditional endpoint yield:

$$\hat{y}_{A \rightarrow P,s} = \frac{\sum_{i \in s} A_i P_i}{\sum_{i \in s} A_i}. \quad (8)$$

This quantity describes the retained pipeline: among executions that satisfy the governed-action construct, what fraction attain the frozen endpoint?

We also report the complete $A \times P$ table. The $A = 0, P = 1$ cell checks pipeline conformance. The $A = 1, P = 0$ cell measures how much upstream action evidence remains below the endpoint.

4.5 Observed-opportunity discovery

Every fixed cell contains eight retained binary outcomes. For signal M , let $m_{c,M}$ be the number of positive outcomes among the eight rows in cell c . For an opportunity budget $k \in \{1, \dots, 8\}$, the probability that a uniformly chosen size- k subset of those eight observed rows contains at least one positive is

$$d_{c,M}(k) = 1 - \frac{\binom{8-m_{c,M}}{k}}{\binom{8}{k}}, \quad (9)$$

with $\binom{n}{k} = 0$ for $n < k$. We average this exact finite-population quantity across cells:

$$D_{M,s}(k) = \frac{1}{|C_s|} \sum_{c \in C_s} d_{c,M}(k). \quad (10)$$

We call Equation (10) *observed-opportunity discovery*. It re-expresses the eight retained outcomes without assuming an order. Because all cells have the same eight repetitions, $D_{M,s}(1)$ equals row-level prevalence and $D_{M,s}(8)$ equals the fraction of cells with any observed positive. The full $k = 1, \dots, 8$ table and an ordered-prefix sensitivity appear in Section B.

4.6 Support profiles

For signal M , a cell, task, or task family is positive when it contains at least one row with $M_i = 1$. The support profile in Equation (4) reports the resulting four counts. Denominators remain visible in all tables. Row counts describe event frequency; cell counts describe repeated-condition coverage; task and family counts describe breadth over the retained task surface.

We also inspect concentration within the small set of positive tasks. This step records how many pass rows each supporting task contributes. It adds no model comparison and no task-removal rule.

4.7 Endpoint controls

The endpoint audit uses three existing control types on fresh task states under the frozen verifier:

1. an exact reference patch, expected to pass;
2. an unmodified no-op state, expected to fail;
3. a deterministic reference-derived mutant, expected to fail.

The retained audit contains 33 attempts. Twelve stopped at a host-mount boundary before verifier entry. The remaining 21 runs cover two tasks and two task families. Mutants are used only as controlled negative probes; they are not treated as replacements for naturally occurring faults [14]. For each verifier-reached run, the archive retains the expected result, observed result, replay status, environment identity, exit status, and output hashes.

4.8 Artifact lineage

The retained row table links executions to task, cell, repetition, protocol arm, model route, UTC timestamp, and raw and derived measurement fields. All 2,400 rows retain prompt and result-row integrity hashes. Provider-linked metadata—including a server date and a hashed provider request or call identifier—is recoverable for 2,005 rows. Request hashes cover 2,036 rows and response hashes cover 2,005 rows.

Section D reports coverage by stratum and distinguishes canonical row timestamps from provider event timestamps.

All analyses in this paper are deterministic transformations of the retained records. No model generation, patch execution, or verifier call is added by the manuscript analysis. This separation between retrieved evidence and analysis derivatives follows established reproducibility guidance for studies built from evolving software repositories [8].

5 Results

5.1 RQ1: The raw action field contains twelve invariant conflicts

The DeepSeek raw action fields agree with the harmonized construct for all rows. The GLM stratum contains 12 conflicts. Each disputed row has $\bar{A}_i = 1$ and $E_i = 1$, while the retained patch-application check has $G_i = 0$ and the failure record identifies patch application as the stopping step. All twelve rows belong to `fa1-1.3.0-roadmap` and span both protocol arms.

The historical field therefore records 98 positive GLM actions; the harmonized construct records 86. Table 4 shows how the change propagates through the support units. Two cells and one task lose their positive status. The set of positive task families remains unchanged because other tasks in the same family retain supported actions.

Table 4: Effect of the GLM field-integrity reconciliation across support units. The twelve disputed rows remain in the immutable raw data.

Action field	Positive rows	Positive cells	Positive tasks	Positive families
Raw historical field	98	29	20	14
Harmonized construct	86	27	19	14
Change	-12	-2	-1	0

The available artifacts identify the invariant conflict but do not reconstruct the legacy mechanism that produced it. The audit preserves the raw field, records the conflict, and derives A_i through Equation (5). This establishes the field-to-construct mapping used in the remaining analyses.

5.2 RQ2: Governed action has low conditional endpoint yield

Table 5 reports the complete execution-level relation between governed action and verifier pass. No stratum contains a pass with $A_i = 0$. This confirms the expected pipeline ordering for the observed passes. The dominant off-diagonal cell is $A_i = 1, P_i = 0$: 185 rows in the clean stratum, 190 in the extension stratum, and 86 in the GLM boundary stratum.

The clean stratum contains 199 governed actions and 14 passes. Its conditional endpoint yield is 7.04%. The extension stratum contains 204 governed actions and 14 passes, for a yield of 6.86%. The GLM boundary contains 86 harmonized actions and no pass. Execution-level action prevalence is 20.73%, 21.25%, and 17.92%; pass prevalence is 1.46%, 1.46%, and 0.00%.

Table 5: Execution-level relation between the harmonized governed-action construct A and verifier pass P . Conditional endpoint yield is $\Pr(P = 1 \mid A = 1)$ as a descriptive fraction within each stratum.

Stratum	Rows	$A=1$	$P=1$	$A=0, P=1$	$A=1, P=0$	Yield	Pass tasks
DeepSeek clean	960	199	14	0	185	7.04%	2/30
DeepSeek extension	960	204	14	0	190	6.86%	1/30
GLM boundary	480	86	0	0	86	0.00%	0/30

The route-by-arm table in Table 13 shows the same ordering in every retained block: governed-action

prevalence exceeds pass prevalence. The size of the gap varies across the historical conditions, which is consistent with the claim index: route, arm, and task surface remain part of Π even when the same construct labels are used.

The result gives governed action a precise role. It is a successful screen for allowed, applicable source activity under the frozen protocol. In these strata, most rows that clear that screen remain below the verifier endpoint.

5.3 RQ3a: Opportunity amplifies action discovery more than pass discovery

Table 6 reports observed-opportunity discovery at $k \in \{1, 2, 4, 8\}$. At $k = 1$, the values equal execution prevalence. At $k = 8$, they equal the fraction of fixed cells with at least one retained positive.

In the clean stratum, governed-action discovery rises from 20.73% at $k = 1$ to 40.83% at $k = 8$. Verifier-pass discovery rises from 1.46% to 4.17%. In the extension stratum, action discovery rises from 21.25% to 46.67%, while pass discovery moves from 1.46% to 1.67%. The GLM boundary reaches 45.00% action discovery and remains at zero pass discovery.

Table 6: Observed-opportunity discovery. Each entry averages the exact probability that a size- k subset of a cell’s eight retained outcomes contains at least one positive observation.

Stratum	Signal	$k = 1$	$k = 2$	$k = 4$	$k = 8$
DeepSeek clean	Governed action	20.73%	28.10%	35.19%	40.83%
DeepSeek clean	Verifier pass	1.46%	2.41%	3.45%	4.17%
DeepSeek extension	Governed action	21.25%	29.02%	37.44%	46.67%
DeepSeek extension	Verifier pass	1.46%	1.67%	1.67%	1.67%
GLM boundary	Governed action	17.92%	26.90%	36.00%	45.00%
GLM boundary	Verifier pass	0.00%	0.00%	0.00%	0.00%

The opportunity transformation does not change the meaning of either signal. It changes the chance that a fixed cell exposes at least one retained positive under a stated budget. The observed curves therefore answer a matched question for A and P . Across all three strata, additional opportunities expose governed actions over a much larger share of cells than verifier passes.

The extension pass curve reaches its $k = 8$ endpoint by $k = 2$: both positive cells contain enough passing rows that every size-two subset has at least one pass. The flat section reflects concentration within two cells, not broad saturation over the task surface. This link between opportunity and support becomes visible only when the two analyses are reported together.

5.4 RQ3b: Endpoint evidence contracts sharply across support units

Table 7 gives the support profile for each signal. Governed actions occupy 49 of 120 clean cells, 21 of 30 clean tasks, and 11 of 14 clean families. The 14 clean passes occupy five cells, two tasks, and one family. In the extension stratum, governed actions occupy 56 cells, 26 tasks, and 12 of 13 families. Its 14 passes occupy two cells, one task, and one family. The GLM actions span all 14 retained task families but yield no endpoint support.

Table 7: Support profiles across nested reporting units. A cell, task, or family is positive when it contains at least one positive execution row.

Stratum	Signal	Rows	Cells	Tasks	Families
DeepSeek clean	Governed action	199	49/120	21/30	11/14
DeepSeek clean	Verifier pass	14	5/120	2/30	1/14
DeepSeek extension	Governed action	204	56/120	26/30	12/13
DeepSeek extension	Verifier pass	14	2/120	1/30	1/13
GLM boundary	Governed action	86	27/60	19/30	14/14
GLM boundary	Verifier pass	0	0/60	0/30	0/14

Across the two task-ID-disjoint DeepSeek strata, the audit inventory contains 403 action rows supported by 105 cells, 47 tasks, and 16 distinct family identifiers. The 28 passing rows are supported by seven cells, three tasks, and two families. These totals summarize the breadth of the retained evidence; rates remain stratum-specific.

Concentration is also strong within the positive tasks. In the clean stratum, `tpl-3.2.0-roadmap` contributes 13 of 14 passes and `tpl-3.3.0-roadmap` contributes one. In the extension stratum, `prm-5.15.0-roadmap` contributes all 14 passes. Table 14 reports the corresponding action counts. Removing the sole pass-support family from either pool removes all passes in that pool; removing any other family leaves the pass numerator unchanged.

The support profiles change the interpretation of the row totals. Twenty-eight passing executions show that the frozen protocol produced accepted candidates. Seven positive cells show how many repeated conditions generated them. Three tasks and two families describe the breadth of that endpoint evidence over the retained task surface.

5.5 RQ4: The verifier separates the executed controls

All 21 verifier-reached controls produce their expected outcome (Table 8). The exact reference patch passes in all six runs. The unrepaired no-op state fails in all six. All nine reference-derived mutants are rejected. The retained replays contain no disagreement.

Table 8: Verifier-reached endpoint controls. Twelve earlier attempts that stopped at a host-mount boundary are recorded separately as infrastructure events.

Control	Runs	Expected	Expected observed	Replay disagreement
Gold/reference	6	Pass	6/6	0
No-op	6	Fail	6/6	0
Reference-derived mutant	9	Fail	9/9	0

The twelve earlier host-mount events stop before verifier entry and remain outside the endpoint denominator. Their separate accounting keeps infrastructure delivery and behavioral evaluation distinct.

The executed controls cover two tasks and two task families. Within that surface, the verifier distinguishes known-good, unrepaired, and selected near-reference candidates under clean replay. This result supports the frozen pass criterion used in Equation (8); it leaves semantic adequacy beyond the tested controls unmeasured.

6 Implications for Repair Evaluation

6.1 A reported number should carry its claim coordinates

A percentage alone does not identify the question it answers. The extension stratum supports at least four valid summaries: 204/960 execution rows contain a governed action; 56/120 cells contain one by eight

retained opportunities; 26/30 tasks contain one; and 14/960 rows pass the verifier. The values become comparable after their constructs, budgets, and units are aligned.

The claim index provides a short reporting discipline. Authors can attach the five coordinates in prose, a result table, or a machine-readable record. Table 9 gives one example. The added information is small relative to the cost of a repository-level experiment, yet it fixes the denominator, the opportunity transformation, the support unit, and the meaning of the endpoint.

Table 9: Example claim card for the extension-stratum governed-action result. The card keeps the estimate attached to its measurement conditions.

Field	Recorded value
Construct C	Governed action: positive raw action decision with supported deterministic application and patch-application check
Opportunity K	Eight retained executions per fixed task–route–arm cell; subset-averaged any-positive discovery
Support unit U	Fixed cell for the estimate; positive support also reported for tasks and task families
Interpretation boundary V	Observed governed-action discovery; verifier pass and semantic adequacy are separate claims
Instrumentation Π	DeepSeek extension stratum, 30 task-ID-disjoint tasks, two routes, two protocol arms
Estimate	56/120 positive cells, or 46.67% observed-opportunity discovery at $k = 8$
Broader support	26/30 tasks and 12/13 task families contain at least one governed action

The mismatch set makes a numerical contrast explicit. The governed-action and verifier-pass rows in Table 6 share K , U , and Π ; the construct and its interpretation boundary change together. Their gap therefore compares two evidence layers under the same opportunities. A contrast between cell-level action discovery at $k = 8$ and execution-level pass prevalence also changes opportunity and unit. That gap describes a composite difference and cannot isolate one measurement choice.

6.2 Measurement fields require versioned semantics

The twelve GLM conflicts show why a result table needs more than a column name. A stored Boolean inherits meaning from runner logic, scorer logic, event ordering, and retained evidence. The operational invariant belongs in the schema. When the invariant cannot be established, a derived field should record the decision without rewriting the raw history.

Versioned harmonization also needs unit-aware impact accounting. In this study, twelve row-level changes remove two positive cells and one positive task while leaving family support unchanged. Reporting only the revised row count would hide how the correction changes broader evidence. The same principle applies to parser changes, patch-application rules, timeout handling, and verifier-entry classification.

Instrumentation provenance completes the field definition. A task surface, model route, provider snapshot, runner, and scorer jointly determine Π . Hosted models make exact regeneration difficult, but artifact replay and deterministic local-stage recomputation remain available when response, patch, and result lineage is retained. Provider-linked dates and hashed request identifiers add auditability without turning a mutable hosted backend into a fixed experimental object.

6.3 Opportunity and support answer complementary questions

Opportunity measures exposure within a fixed condition. Support measures breadth across conditions. The distinction matters most when a system is stochastic and the endpoint is sparse. Eight attempts can reveal that a model sometimes reaches a usable action surface. The same eight attempts do not create eight new tasks.

The present data show both effects at once. Action discovery expands across the retained opportunity budget, reaching roughly two-fifths to one-half of cells. Endpoint evidence contracts when the unit

broadens: 28 passing rows reduce to seven cells, three tasks, and two families. Neither pattern invalidates the row-level observations. Together they identify where the evidence is abundant and where it is narrow.

This joint view can improve benchmark reports and system papers. A useful minimum is:

1. the per-execution prevalence of each named signal;
2. the any-positive discovery value at the declared opportunity budget;
3. the number of positive cells, tasks, and repositories or task families;
4. the relation between upstream signals and the final operational endpoint.

The four items describe frequency, opportunity, breadth, and criterion relation without collapsing them into one score.

6.4 Five substitutions explain many apparent disagreements

Table 10 summarizes five ways a repair statistic can acquire a stronger meaning than its evidence supplies. The substitutions are analytically distinct. Fixing the opportunity budget does not validate the underlying field. Validating the field does not establish verifier pass. A stable verifier does not by itself establish semantic adequacy.

Table 10: Five common substitutions and the reporting information that resolves them.

Substitution	What is exchanged	Required report
Field	Stored flag for its intended construct	Raw field, operational invariant, derived rule, and affected units
Construct	Upstream activity for a stronger behavioral endpoint	Matched construct names, $A \times P$ relation, and conditional endpoint yield
Opportunity	A multi-attempt discovery value for a single-attempt rate	Number of attempts, selection rule, aggregation rule, and matched endpoint budget
Support	Repeated positive rows for broad task evidence	Positive rows, cells, tasks, and repositories or families
Oracle	Frozen verifier pass for semantic adequacy	Verifier definition, controls, replay behavior, and stated interpretation boundary

The table also suggests a reviewable artifact standard. Each headline result can point to the raw column, construct rule, aggregation script, support table, verifier specification, and artifact manifest that sustains it. This creates a trace from claim to evidence rather than a longer list of generic reproducibility materials.

6.5 Intermediate signals retain a clear engineering role

Governed action is useful precisely because it sits upstream of pass. It distinguishes empty or invalid model output from a concrete source action that reaches strict application. A system with low governed-action prevalence has a different bottleneck from one with frequent actions and low endpoint yield. The claim index preserves that diagnostic information and keeps the repair endpoint separate.

The same distinction applies when an intermediate signal is used for search, ranking, or feedback. Its association with pass describes a measurement relation. Estimating its value to a controller requires an intervention that changes signal access or selection while holding the relevant budget and task conditions fixed. The current audit supplies the constructs and reporting units needed for such a study.

7 Threats to Validity

Task and model scope. The empirical magnitudes come from frozen, admission-filtered RoadmapBench-derived task pools, two historical DeepSeek route labels, and one GLM route. The 60 DeepSeek task

IDs are disjoint across the clean and extension pools; the GLM stratum reuses the clean tasks. Results may differ under other repositories, stronger models, agent loops, edit interfaces, and verifiers. Prior APR and live-code studies show that benchmark composition and time can materially affect observed performance [6, 10]. The claim-indexed framework is broader than these magnitudes because the same construct, opportunity, support, and endpoint choices arise across repository-repair systems.

Historical instrumentation. The study reconstructs measurement semantics from retained records. Twelve GLM rows lack the original artifact needed to identify the exact legacy mechanism behind the action/apply conflict. The harmonized field applies a documented invariant and preserves the disputed raw values. Other unobserved historical inconsistencies may remain where the artifact chain is incomplete. The retained reconciliation table makes the known boundary inspectable.

Support units. Execution rows share cells and tasks, and task instances may share repositories or technical structure. The retained records provide task-family identifiers but no reliable repository identifier. Support profiles therefore stop at task family. Family labels are useful for concentration analysis, though they are an imperfect substitute for repository-level clustering. The paper uses support counts descriptively and avoids row-level independence assumptions.

Opportunity estimand. Observed-opportunity discovery is an exact re-expression of eight retained outcomes per cell. It does not estimate a future provider’s sampling distribution or the behavior of an adaptive best-of- k policy. Its purpose is a matched within-corpus comparison of how two signals become visible as the retained opportunity budget increases.

Endpoint scope. The control audit covers 21 verifier-reached runs across two tasks and two task families. It samples three useful control classes but leaves many incorrect-patch forms untested. Generated-test and patch-assessment studies demonstrate both the value and incompleteness of stronger automated checks [17, 40, 47]. The result supports local discrimination on these controls. Semantic adequacy, developer intent, and behavior beyond the frozen test surface remain outside the observed endpoint.

Hosted-model provenance. Provider-linked metadata is available for 2,005 of 2,400 rows, while prompt and result-row hashes cover the full corpus. These records support request-level chronology and artifact linkage. They cannot recover opaque provider weights or guarantee bitwise regeneration. The paper treats a hosted route and access period as part of the instrumentation stratum.

8 Conclusion

Repository-level repair evidence is produced through a sequence of measurements. A logged action field must satisfy its operational invariant; a governed action must still reach the verifier; repeated opportunities must be separated from broader task support; and a verifier pass retains an explicit semantic boundary.

The 2,400-execution audit shows why these distinctions matter. Twelve raw action positives fail the application invariant used by the governed-action construct. After harmonization, conditional endpoint yield is 7.04% and 6.86% in the two DeepSeek strata. At eight retained opportunities, action discovery reaches 40.83% and 46.67%, while pass discovery reaches 4.17% and 1.67%. The 28 passing rows occupy seven cells, three tasks, and two task families. Endpoint controls confirm the expected behavior on the tested reference, no-op, and mutant cases.

The claim index binds each result to construct, opportunity design, support unit, endpoint interpretation, and instrumentation provenance. This specification keeps repair statistics comparable and makes intermediate evidence useful on its own terms. A number becomes a scientific result when the claim it measures is equally explicit.

Data and Artifact Availability

The public reproduction package is maintained at <https://github.com/Jiangwei-Xue/paper-03-1lm-repair-evidence-ladder>. It provides the manuscript source; the canonical 2,400-row claim table and 300 fixed-cell view; deterministic analysis code for rebuilding the reported tables; frozen protocols, configurations, and task manifests; the public H5/E5/VCR row ledgers and hash relationships; full-transcript prompt-visibility audit reports; provenance and release records; retained configuration metadata for 60 task-specific Docker images; and the experiment harness, live runners, strict edit-transport implementation, answer-state contract, and local contract tests needed to inspect the recorded evaluation path. The package also supplies documented offline commands for validating the evidence chain, regenerating claim-bearing analyses, and recompiling this manuscript.

The release does not redistribute Docker image layers, third-party repository snapshots, protected verifier material, private provider payloads, hosted-model weights, or credentials. The public E5/VCR records establish the released row-level relationships and integrity boundaries, but omitted private artifacts cannot all be dereferenced from the public package. Accordingly, the release supports deterministic offline evidence replay, analysis, and source-level inspection; it does not claim bitwise regeneration of historical hosted-model outputs or a self-contained fresh provider-and-Docker rerun.

References

- [1] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785.
- [2] L. C. Briand, S. Morasca, and V. R. Basili. Property-based software engineering measurement. *IEEE Transactions on Software Engineering*, 22(1):68–86, 1996. doi: 10.1109/32.481535.
- [3] W. Caban. Measurement without validity: The compounding reliability problem in agentic AI evaluation. *arXiv preprint arXiv:2608.00794*, 2026. URL <https://arxiv.org/abs/2608.00794>.
- [4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [5] Z. Chen, Z. Sun, Y. Shi, C. Peng, X. Gu, D. Lo, and L. Jiang. Rethinking the value of agent-generated tests for LLM-based software engineering agents. *arXiv preprint arXiv:2602.07900*, 2026. URL <https://arxiv.org/abs/2602.07900>.
- [6] T. Durieux, F. Madeiral, M. Martinez, and R. Abreu. Empirical review of java program repair tools: A large-scale experiment on 2,141 bugs and 23,551 repair attempts. In *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 2019. doi: 10.1145/3338906.3338911.
- [7] L. Gazzola, D. Micucci, and L. Mariani. Automatic software repair: A survey. *IEEE Transactions on Software Engineering*, 45(1):34–67, 2019. doi: 10.1109/TSE.2017.2755013.
- [8] J. M. González-Barahona and G. Robles. On the reproducibility of empirical software engineering studies based on data retrieved from development repositories. *Empirical Software Engineering*, 17(1–2):75–89, 2012. doi: 10.1007/s10664-011-9181-9.
- [9] J. He, M. Shi, J. Jia, X. Yang, and D. Sun. What process evaluation of coding agents actually measures: Action, task, and step are three different levels. *arXiv preprint arXiv:2608.22960*, 2026. URL <https://arxiv.org/abs/2608.22960>.

- [10] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination-free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/94074dd5a072d28ff75a76dabed43767-Abstract-Conference.html.
- [11] J. Jiang, S. Zheng, N. Vidra, and S. Setty. Beyond pass@k: Measuring reliability and security of agentic code generation. *arXiv preprint arXiv:2608.14711*, 2026. URL <https://arxiv.org/abs/2608.14711>.
- [12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [13] R. Just, D. Jalali, and M. D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis*, pages 437–440. ACM, 2014. doi: 10.1145/2610384.2628055.
- [14] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 654–665. ACM, 2014. doi: 10.1145/2635868.2635929.
- [15] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, and J. Rosenberg. Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, 28(8):721–734, 2002. doi: 10.1109/TSE.2002.1027796.
- [16] X.-B. D. Le, L. Bao, D. Lo, X. Xia, S. Li, and C. S. Pasareanu. On reliability of patch correctness assessment. In *2019 IEEE/ACM 41st International Conference on Software Engineering*, pages 524–535. IEEE, 2019. doi: 10.1109/ICSE.2019.00064.
- [17] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract.html.
- [18] F. Long and M. C. Rinard. An analysis of the search spaces for generate and validate patch generation systems. In *Proceedings of the 38th International Conference on Software Engineering*, pages 702–713. ACM, 2016. doi: 10.1145/2884781.2884872.
- [19] W. Ma, Z. Chen, J. Gu, T. Li, S. Liu, and L. Jiang. Same signal, different semantics: A cross-framework behavioral analysis of software engineering agents. *arXiv preprint arXiv:2605.18332*, 2026. URL <https://arxiv.org/abs/2605.18332>.
- [20] S. Miserendino, M. Wang, T. Patwardhan, and J. Heidecke. SWE-Lancer: Can frontier LLMs earn \$1 million from real-world freelance software engineering? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 44412–44450, 2025. URL <https://proceedings.mlr.press/v267/miserendino25a.html>.
- [21] M. Monperrus. Automatic software repair: A bibliography. *ACM Computing Surveys*, 51(1): 17:1–17:24, 2018. doi: 10.1145/3105906.
- [22] Z. Qi, F. Long, S. Achour, and M. C. Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, pages 24–36. ACM, 2015. doi: 10.1145/2771783.2771791.

- [23] P. Ralph and E. D. Tempero. Construct validity in software engineering research and software metrics. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering*, pages 13–23. ACM, 2018. doi: 10.1145/3210459.3210461.
- [24] P. Ralph, N. b. Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, et al. Empirical standards for software engineering research. *arXiv preprint arXiv:2010.03525*, 2020. URL <https://arxiv.org/abs/2010.03525>.
- [25] M. S. Rashid, C. Bock, Y. Zhuang, A. Buchholz, T. Esler, S. Valentin, L. Franceschi, M. Wistuba, P. T. Sivaprasad, W. J. Kim, A. Deoras, G. Zappella, and L. Callot. SWE-PolyBench: A multi-language benchmark for repository-level evaluation of coding agents. *arXiv preprint arXiv:2504.08703*, 2025. URL <https://arxiv.org/abs/2504.08703>.
- [26] P. Sahoo, G. Mittal, X. Li, S. Ma, B. Steenhoeck, P. Lin, and Y. Hu. AgentLens: Revealing the lucky pass problem in SWE-agent evaluation. *arXiv preprint arXiv:2605.12925*, 2026. URL <https://arxiv.org/abs/2605.12925>.
- [27] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875.
- [28] J. Shao, H. Chen, W. Zhang, M. Pan, and B. Luo. Do agent benchmarks measure capability? protocol validity in the age of agentic AI. *arXiv preprint arXiv:2607.22368*, 2026. URL <https://arxiv.org/abs/2607.22368>.
- [29] R. Shu, C. Y. Chong, X. Zhou, Y. Peng, Z. Wu, X. Han, Z. Zhuang, G. Yuan, and Y. Wang. What resolve rate hides: Trajectory structure diagnostics for coding agents. *arXiv preprint arXiv:2607.06184*, 2026. URL <https://arxiv.org/abs/2607.06184>.
- [30] D. I. K. Sjøberg and G. R. Bergersen. Construct validity in software engineering. *IEEE Transactions on Software Engineering*, 49(3):1374–1396, 2023. doi: 10.1109/TSE.2022.3176725.
- [31] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 532–543. ACM, 2015. doi: 10.1145/2786805.2786825.
- [32] M. V. T. Thai, T. Le, D. N. Manh, H. P. Nhat, and N. D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025. URL <https://arxiv.org/abs/2512.18470>.
- [33] B. Wang, C. Zhang, D. Liu, J. Zhang, J. Chen, M. Li, M. Chen, R. Fang, S. Zhang, X. Wang, Y. Jing, Z. Ma, and Z. Cui. The verification horizon: No silver bullet for coding agent rewards. *arXiv preprint arXiv:2606.26300*, 2026. URL <https://arxiv.org/abs/2606.26300>.
- [34] S. Wang, M. Wen, B. Lin, H. Wu, Y. Qin, D. Zou, X. Mao, and H. Jin. Automated patch correctness assessment: How far are we? In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, pages 968–980. ACM, 2020. doi: 10.1145/3324884.3416590.
- [35] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=0Jd3ayDDoF>.
- [36] Y. Wang, M. Pradel, and Z. Liu. Are “solved issues” in SWE-bench really solved correctly? an empirical study. *arXiv preprint arXiv:2503.15223*, 2025. URL <https://arxiv.org/abs/2503.15223>.

- [37] Y. Wang, X. Han, D. Shang, Y. Tang, and B. Liu. Safety, or just capability? a validity audit of agent-safety benchmarks. *arXiv preprint arXiv:2607.28685*, 2026. URL <https://arxiv.org/abs/2607.28685>.
- [38] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi: 10.1007/978-3-642-29044-2.
- [39] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024. URL <https://arxiv.org/abs/2407.01489>.
- [40] Q. Xin and S. P. Reiss. Identifying test-suite-overfitted patches through test case generation. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 226–236. ACM, 2017. doi: 10.1145/3092703.3092718.
- [41] Y. Xiong, X. Liu, M. Zeng, L. Zhang, and G. Huang. Identifying patch correctness in test-based program repair. In *Proceedings of the 40th International Conference on Software Engineering*, pages 789–799. ACM, 2018. doi: 10.1145/3180155.3180182.
- [42] X. Xu and W. Wu. Validation evidence in LLM repair agents: How much of what passes actually tests the bug? *arXiv preprint arXiv:2607.28871*, 2026. URL <https://arxiv.org/abs/2607.28871>.
- [43] X. Xu, R. Yang, H. Shen, W. Xu, B. Gao, R. Wu, K. Shi, W. Xie, X. Chen, M. Wu, J. Zeng, M. Heinrich, E. Zhang, L. Chen, K. Li, and B. Chang. Roadmapbench: Evaluating long-horizon agentic software development across version upgrades. *arXiv preprint arXiv:2605.15846*, 2026. URL <https://arxiv.org/abs/2605.15846>.
- [44] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL <https://openreview.net/forum?id=m06mZAo3de>.
- [45] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. SWE-smith: Scaling data for software engineering agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-3239.
- [46] H. Ye, M. Martinez, T. Durieux, and M. Monperrus. A comprehensive study of automatic program repair on the QuixBugs benchmark. *Journal of Systems and Software*, 157:110825, 2019. doi: 10.1016/j.jss.2019.110825. Article 110825.
- [47] H. Ye, M. Martinez, and M. Monperrus. Automated patch assessment for program repair at scale. *Empirical Software Engineering*, 26(2):20, 2021. doi: 10.1007/s10664-020-09920-w.
- [48] J. Yi, S. H. Tan, S. Mehtaev, M. Böhme, and A. Roychoudhury. A correlation study between automated program repair and test-suite metrics. *Empirical Software Engineering*, 23(5):2948–2979, 2018. doi: 10.1007/s10664-017-9552-y.
- [49] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang. SWE-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025. URL <https://arxiv.org/abs/2505.23419>.
- [50] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1592–1604. ACM, 2024. doi: 10.1145/3650212.3680384.

A Operational Definitions and Claim Index Examples

Table 11: Operational variables used in the claim-bearing analysis.

Variable	Positive rule	Interpretation
Raw action $\tilde{A}_i$	Historical action flag equals 1	The historical scorer recorded a positive action decision
Application E_i	Deterministic edit application equals 1	The explicit edit operations passed the deterministic application stage
Patch check G_i	Patch-application check equals 1	The retained patch-application check succeeded
Governed action A_i	$\tilde{A}_i E_i G_i = 1$	A positive action decision supported by the retained application evidence
Verifier pass P_i	Historical endpoint field equals 1	The candidate met the frozen local endpoint
Semantic adequacy Y_i	No complete operational measure	The patch satisfies the intended behavior beyond the observed verifier surface

Schema keys are action_governance_success, deterministic_edit_apply_success, patch_apply_check_ok, and answer_success.

A.1 Positive-event coding

The canonical source contains historical fields with different missingness patterns. The claim-bearing indicators in Table 11 use a positive-event rule: a value contributes one only when the required retained fields are explicitly positive. Every planned row remains in the denominator. This coding answers how often a positive construct was observed under the planned execution budget. It does not classify every nonpositive row as an explicit behavioral failure.

For A_i , the retained harmonized field is complete over the 2,400 rows. The twelve GLM conflicts have $\tilde{A}_i = 1$, $E_i = 1$, and $G_i = 0$, so Equation (5) assigns $A_i = 0$. The reconciliation table retains the raw field and records the source path and reason for each derived value.

A.2 Examples of indexed estimates

The following examples show how the same corpus yields distinct, well-specified results:

$$\begin{aligned}
 \mathcal{I}(204/960) &= \langle \text{governed action}, 1, \text{row}, \text{applicable-action claim}, \text{extension} \rangle, \\
 \mathcal{I}(56/120) &= \langle \text{governed action}, k=8 \text{ any-positive}, \text{cell}, \text{applicable-action claim}, \text{extension} \rangle, \\
 \mathcal{I}(14/960) &= \langle \text{verifier pass}, 1, \text{row}, \text{frozen-pass claim}, \text{extension} \rangle, \\
 \mathcal{I}(1/30) &= \langle \text{verifier pass}, \text{all retained rows}, \text{task}, \text{frozen-pass claim}, \text{extension} \rangle.
 \end{aligned}$$

The first two differ in opportunity and unit. The first and third differ in construct. The third and fourth summarize the same endpoint at different support units.

A.3 Broader event vocabulary

The historical project also records structured-state validity, deterministic edit application, verifier entry, partial reward, and infrastructure states where available. These events remain useful for row-level diagnosis. Their coverage is uneven across the three historical strata, so the paper does not construct a cross-stratum funnel from them. The retained row table preserves the available fields for future task-local analyses.

B Full Opportunity Analysis

Table 12: Full observed-opportunity discovery table (percent). A denotes harmonized governed action and P denotes verifier pass.

k	Clean A	Clean P	Extension A	Extension P	GLM A	GLM P
1	20.73	1.46	21.25	1.46	17.92	0.00
2	28.10	2.41	29.02	1.67	26.90	0.00
3	32.34	3.04	33.84	1.67	32.29	0.00
4	35.19	3.45	37.44	1.67	36.00	0.00
5	37.28	3.74	40.31	1.67	38.90	0.00
6	38.84	3.93	42.71	1.67	41.31	0.00
7	40.00	4.06	44.79	1.67	43.33	0.00
8	40.83	4.17	46.67	1.67	45.00	0.00

B.1 Relation to ordered prefixes

The main estimand averages all size- k subsets of the eight retained outcomes in each cell. The retained analysis records also support an ordered-prefix sensitivity:

$$D_{M,s}^{\text{prefix}}(k) = \frac{1}{|C_s|} \sum_{c \in C_s} \mathbf{1}\left(\max_{1 \leq r \leq k} M_{c,r} = 1\right).$$

The prefix form depends on the retained repetition order. The subset-averaged form removes that ordering choice and is therefore the primary analysis. Both forms use the same eight historical rows and converge to the same $k = 8$ endpoint.

B.2 Interpretation

Observed-opportunity discovery is a finite-corpus quantity. It can be reproduced exactly from the retained cell table. It does not model adaptive selection, correlated future provider outputs, or additional attempts beyond eight. A prospective reliability study would require a sampling design that treats future rollouts as the target population.

C Stratified and Concentration Sensitivity

Table 13: Descriptive route-by-arm summaries. The historical routes and prompt arms define measurement strata; the table is not a causal comparison of methods.

Pool	Route	Arm	Rows	Actions	Passes	Yield
DeepSeek clean	flash	single-shot	240	51 (21.25%)	4 (1.67%)	7.84%
DeepSeek clean	flash	staged	240	34 (14.17%)	3 (1.25%)	8.82%
DeepSeek clean	pro	single-shot	240	56 (23.33%)	2 (0.83%)	3.57%
DeepSeek clean	pro	staged	240	58 (24.17%)	5 (2.08%)	8.62%
DeepSeek extension	flash	single-shot	240	60 (25.00%)	7 (2.92%)	11.67%
DeepSeek extension	flash	staged	240	41 (17.08%)	0 (0.00%)	0.00%
DeepSeek extension	pro	single-shot	240	61 (25.42%)	7 (2.92%)	11.48%
DeepSeek extension	pro	staged	240	42 (17.50%)	0 (0.00%)	0.00%
GLM boundary	glm-5.2	single-shot	240	40 (16.67%)	0 (0.00%)	0.00%
GLM boundary	glm-5.2	staged	240	46 (19.17%)	0 (0.00%)	0.00%

All ten route-by-arm blocks contain more governed actions than verifier passes. The extension passes occur in the single-shot arm for both recorded DeepSeek routes; the staged arm has no pass in that task pool. These values describe historical conditions and are not estimates of an arm effect.

Table 14: Task instances supporting verifier passes.

Stratum	Task	Pass rows	Action rows	Share of stratum passes
DeepSeek clean	tp1-3.2.0-roadmap	13	24	92.86%
DeepSeek clean	tp1-3.3.0-roadmap	1	5	7.14%
DeepSeek extension	prm-5.15.0-roadmap	14	16	100.00%

The clean passes all belong to the tp1 family. The extension passes all belong to the prm family. In the leave-one-family-out analysis, excluding the corresponding positive family sets the pool’s pass count to zero; excluding any other family preserves the pass numerator. This result follows directly from the support distribution in Table 14.

The retained analysis records also contain equal-family bootstrap summaries for action prevalence, pass prevalence, and task-level action-to-pass yield. Their intervals are descriptive sensitivity summaries over the retained families. The main text emphasizes observed support counts because the task pools were frozen evidence sets rather than draws from a defined family population.

D Provenance and Endpoint Coverage

Table 15: Coverage of retained provenance fields. Every provider-linked row contains a server date and a hashed provider call or request identifier.

Stratum	Rows	Provider-linked	Request hash	Response hash	Prompt hash	Result hash
DeepSeek clean	960	803	803	803	960	960
DeepSeek extension	960	819	819	819	960	960
GLM boundary	480	383	414	383	480	480

Provider-linked coverage is partial and explicit. The DeepSeek clean and extension strata map 803 and 819 rows, respectively, to retained provider response metadata. The GLM boundary maps 383 rows. Each mapped row includes a server date and a hashed provider request or call identifier; DeepSeek mappings also include hashed trace identifiers. Request-hash coverage is slightly broader in the GLM stratum than response-metadata coverage. Prompt and final result-row hashes cover every row.

Table 16: Canonical result-row timestamp windows. Provider event timestamps are retained separately where available.

Stratum	First retained row	Last retained row
DeepSeek clean	2026-07-07 10:19:29.455569 UTC	2026-07-07 19:35:37.038631 UTC
GLM boundary	2026-07-08 14:34:27.950363 UTC	2026-07-09 11:03:54.659482 UTC
DeepSeek extension	2026-07-10 06:34:40.472569 UTC	2026-07-10 10:14:53.033011 UTC

Table 16 reports canonical result-row timestamps. A provider’s server date identifies the service event; the canonical timestamp identifies the retained result record. The archive stores them as separate events.

D.1 Endpoint-control accounting

The endpoint audit records 33 attempts. Twelve initial attempts encounter a host-mount permission problem before verifier entry. Corrected fresh-path attempts yield 21 verifier-reached outcomes: six

reference runs, six no-op runs, and nine mutant runs. The two task instances are `fa1-1.3.0-roadmap` and `tpl-3.2.0-roadmap`. The former contributes three reference, three no-op, and six mutant runs; the latter contributes three of each reference and no-op plus three mutants.

The retained control table contains control type, expected and observed outcome, verifier-entry state, infrastructure classification, replay disagreement, exit status, runtime, environment identity, and hashes for retained outputs. Protected verifier material is stored separately.

D.2 Development chronology

The project archive retains two design records with July 5, 2026 filesystem metadata, their normalized hashes and timestamp records, canonical execution records from July 7–10, provider-linked service metadata for mapped rows, and local Git records from July 10. Timestamp types are normalized separately and labeled by trust class.